UNIVERSITÀ DEGLI STUDI
DI SALERNO

Laurea magistrale ingegneria informatica

Sistemi Embedded 2024/2025 - Gruppo 17

Sistema embedded per la domotica

Report progetto

Cipriano Ivan Luigi	0622702507
Faraulo Simone	0622702509

Indice

1 Introduzione **2**

2 Definizione dei requisiti **3**

2.1 User Stories 3

3 Definizione degli Use Cases **8**

4 Progettazione **11**

4.1 L'ambiente di progettazione `Simulink Stateflow` 11

4.2 Input del sistema 12

4.3 Output del sistema 13

4.4 Struttura del firmware in Stateflow 14

4.4.1 Stato SistemaInattivo 14

4.4.2 Stato SistemaAttivo 16

4.4.3 Stato ConfigurazioneDispositivo 17

4.4.4 Stato EsecuzioneDispositivo 21

4.5 Funzioni Matlab Implementate 23

4.5.1 Funzione `verificaValoreLuci` 23

4.5.2 Funzione `conversioneNum` 24

4.5.3 Funzione `verificaValoreVeneziane` 24

4.5.4 Funzione `conversioneValoreTemporizzazione` 25

4.5.5 Funzione `gestisciBufferInput` 25

4.5.6 Funzione `conversioneValoreTemperatura` 25

4.5.7 Funzione `cambioDuty` 26

4.5.8 Funzione `dutyVeneziane` 26

4.5.9 Funzione `codifica_ascii_car_speciali` 26

1 Introduzione

Il progetto ha come obiettivo lo sviluppo di un sistema embedded per la gestione integrata di un ambiente domestico, in grado di controllare in maniera coordinata diversi dispositivi. In particolare, il sistema permette la regolazione dell'apertura delle veneziane, l'accensione e lo spegnimento delle luci (anche attraverso funzionalità di temporizzazione), nonché la regolazione del riscaldamento.

L'interazione con il sistema avviene attraverso due modalità distinte ma complementari. Da un lato è disponibile un'interfaccia UART, che consente all'utente di inserire una configurazione personalizzata e di monitorare in tempo reale lo stato del sistema. Dall'altro lato è possibile interagire mediante pulsanti fisici, che garantiscono un controllo diretto delle funzionalità principali anche in assenza di collegamento seriale. A supporto di queste modalità di interazione, è stata inoltre implementata una serie di indicatori LED che forniscono un riscontro visivo immediato sullo stato operativo del sistema.

L'analisi dei requisiti è stata condotta seguendo un approccio di tipo agile, con l'impiego delle user stories come strumento principale per la raccolta e la formalizzazione delle esigenze. In questo modo è stato possibile tradurre in requisiti tecnici le necessità prioritarie dell'utente finale, garantendo che lo sviluppo del sistema fosse guidato da obiettivi concreti e verificabili.

Per quanto riguarda la fase di progettazione, la macchina a stati finiti è stata modellata in ambiente `Simulink`, facendo uso dell'estensione `Stateflow` per la definizione delle logiche di controllo. Successivamente, il codice generato automaticamente è stato integrato sulla scheda di sviluppo `STM32 Nucleo G474RE`, consentendo di effettuare test e verifiche dirette sui requisiti precedentemente definiti.

2 Definizione dei requisiti

In questa sezione sono riportate le User Stories del sistema, con le relative descrizioni e i criteri di accettazione. Ogni User Story illustra una funzionalità dal punto di vista dell'utente e specifica le condizioni che devono essere soddisfatte per considerare la funzionalità correttamente implementata.

2.1 User Stories

ID	US1
Titolo	Attivazione del sistema
User Story	**As** utente, **I want** poter avviare il sistema, **so that** sia possibile accedere alla fase di configurazione iniziale.
Criteri di Accettazione	• **Given** il sistema in stato inattivo, **When** l'utente preme il pulsante di **ATTIVAZIONE**, **Then** il LED di stato UTENTE deve lampeggiare con una frequenza costante di 2 Hz. • **Given** il sistema in stato inattivo, **When** l'utente preme il pulsante di **ATTIVAZIONE**, **Then** sull'interfaccia UART deve essere visualizzato il messaggio di ingresso in modalità di configurazione.

ID	US2
Titolo	Disattivazione del sistema
User Story	**As** utente, **I want** poter disattivare il sistema, **so that** sia possibile arrestarne in maniera sicura il funzionamento e riportare tutti i dispositivi collegati allo stato di riposo.
Criteri di Accettazione	• **Given** il sistema in stato di funzionamento o in modalità di configurazione, **When** l'utente preme il pulsante di **ATTIVAZIONE**, **Then** il sistema deve passare allo stato inattivo, spegnendo il LED di stato UTENTE e impostando le uscite nei seguenti valori di default: veneziane completamente chiuse, luci spente e riscaldamento disattivato con temperatura impostata al minimo.

ID	US3
Titolo	Configurazione del sistema
User Story	**As** utente, **I want** accedere alla modalità di configurazione, **so that** possa modificare e impostare correttamente i parametri operativi del sistema.
Criteri di Accettazione	• **Given** il sistema attivo in modalità di funzionamento, **When** l'utente preme il pulsante **CONFIG**, **Then** il dispositivo deve consentire l'ingresso. • **Given** il sistema in modalità di configurazione per la prima volta, **When** l'utente preme il pulsante **CONFIG**, **Then** l'uscita non è consentita fino al completamento della configurazione iniziale. • **Given** il sistema in modalità di configurazione ed è già presente una configurazione valida per il dispositivo, **When** l'utente preme il pulsante **CONFIG**, **Then** il sistema si porta in modalità funzionamento. • **Given** il sistema in modalità di configurazione, **When** l'utente inserisce un valore non valido, **Then** l'interfaccia UART deve mostrare un messaggio di errore e permettere il reinserimento. • **Given** il sistema in modalità di configurazione, **When** l'utente inserisce correttamente tutti i valori richiesti, **Then** il sistema deve passare automaticamente alla modalità di funzionamento.

ID	US4
Titolo	Regolazione automatizzata delle veneziane
User Story	**As** utente, **I want** poter regolare l'angolo di apertura delle veneziane, **so that** sia possibile modulare con precisione l'ingresso della luce naturale all'interno dell'ambiente domestico.
Criteri di Accettazione	• **Given** il sistema in modalità di configurazione, **When** l'utente seleziona un valore di apertura compreso tra 0° e 180° con step di regolazione pari a 20°, **Then** al termine della configurazione il servomotore deve posizionare le veneziane all'angolazione impostata.

ID	US5
Titolo	Regolazione temperatura
User Story	**As** utente, **I want** poter impostare una temperatura desiderata, **so that** rendere confortevole l'ambiente.
Criteri di Accettazione	• **Given** il sistema in modalità di configurazione, **When** l'utente seleziona un valore di temperatura compreso tra 16°C e 30°C, **Then** al termine della configurazione viene impostata la temperatura inserita. • **Given** il sistema in modalità di configurazione, **When** l'utente inserisce un valore di temperatura tra 16°C e 18°C, **Then** al termine della conf. la temperatura è impostata e il colore del led RGB è **BLU**. • **Given** il sistema in modalità di configurazione, **When** l'utente inserisce un valore di temperatura tra 19°C e 21°C, **Then** al termine della conf. la temperatura è impostata e il colore del led RGB è **VIOLA**. • **Given** il sistema in modalità di configurazione, **When** l'utente inserisce un valore di temperatura tra 22°C e 24°C, **Then** al termine della conf. la temperatura è impostata e il colore del led RGB è **GIALLO**. • **Given** il sistema in modalità di configurazione, **When** l'utente inserisce un valore di temperatura tra 25°C e 27°C, **Then** al termine della conf. la temperatura è impostata e il colore del led RGB è **ARANCIONE**. • **Given** il sistema in modalità di configurazione, **When** l'utente inserisce un valore di temperatura tra 28°C e 30°C, **Then** al termine della conf. la temperatura è impostata e il colore del led RGB è **ROSSO**.

ID	US6
Titolo	Visualizzazione dello stato del sistema
User Story	**As** utente, **I want** poter visualizzare in tempo reale lo stato del sistema, **so that** sia possibile monitorarne e verificarne il corretto funzionamento.
Criteri di Accettazione	• **Given** il sistema in modalità di funzionamento, **When** l'utente accede all'interfaccia UART, **Then** deve essere mostrato lo stato aggiornato del sistema con una frequenza di aggiornamento pari a 10 secondi.

ID	US7
Titolo	Controllo luci tramite interfaccia UART
User Story	**As** utente, **I want** poter accendere e spegnere le luci attraverso l'interfaccia UART, **so that** sia possibile gestire l'illuminazione anche da remoto.
Criteri di Accettazione	• **Given** il sistema in modalità di configurazione, **When** l'utente sceglie di spegnere le luci, **Then** al termine della configurazione esse devono risultare spente. • **Given** il sistema in modalità di configurazione, **When** l'utente sceglie di accendere le luci, **Then** al termine della configurazione esse devono risultare accese.

ID	US8
Titolo	Gestione temporizzazione luci tramite UART
User Story	**As** utente, **I want** poter impostare la temporizzazione delle luci, **so that** sia possibile ottimizzare il consumo energetico.
Criteri di Accettazione	• **Given** il sistema in modalità di configurazione, **When** l'utente imposta un valore di temporizzazione compreso tra 5 e 30 secondi, **Then** il periodo di accensione delle luci deve essere configurato secondo il valore inserito. • **Given** il sistema in modalità di configurazione, **When** l'utente imposta un valore di temporizzazione pari a 0 secondi, **Then** il periodo di accensione delle luci deve essere nullo.

ID	US9
Titolo	Controllo luci manuale
User Story	**As** utente, **I want** poter controllare le luci tramite pulsante, **so that** posso accenderle e spegnerle in qualsiasi momento.
Criteri di Accettazione	• **Given** il sistema in modalità di funzionamento e le luci sono spente, **When** l'utente preme il pulsante **LUCI**, **Then** le luci vengono accese. • **Given** il sistema in modalità di funzionamento e le luci sono accese, **When** l'utente preme il pulsante **LUCI**, **Then** le luci vengono spente. • **Given** il sistema in modalità di funzionamento e le luci sono accese in modalità temporizzata, **When** l'utente preme il pulsante **LUCI**, **Then** le luci vengono spente in anticipo.

ID	US10
Titolo	Gestione temporizzazione luci manuale
User Story	**As** utente, **I want** poter controllare le luci in maniera temporizzata tramite pulsante, **so that** posso impostare l'illuminazione temporizzata quando voglio.
Criteri di Accettazione	• **Given** il sistema in modalità di funzionamento e le luci sono spente, **When** l'utente preme il pulsante **LUCI TMP**, **Then** le luci vengono accese per il periodo impostato e poi spente. • **Given** il sistema in modalità di funzionamento e le luci sono accese in modalità non temporizzata, **When** l'utente preme il pulsante **LUCI TMP**, **Then** le luci restano accese per il periodo impostato e poi spente. • **Given** il sistema in modalità di funzionamento e le luci sono accese in modalità temporizzata, **When** l'utente preme il pulsante **LUCI TMP**, **Then** non accade nulla.

3 Definizione degli Use Cases

In questa sezione viene fornita una panoramica dettagliata delle interazioni tra l'utente e il sistema tramite l'utilizzo di Use cases.

ID	UC1
Titolo	Attivazione del sistema
Attori	Utente e Sistema
Eventi	1. L'utente preme il pulsante `ATTIVA` 2. Il `Led Utente` lampeggia ad una frequenza di 2Hz
Pre-Condizioni	Il dispositivo è nello stato `INATTIVO`
Post-Condizioni	Il dispositivo si porta nello stato `ATTIVO` e in modalità `CONFIGURAZIONE` e il sistema mostra il menù di configurazione tramite interfaccia UART
Scenari Alternativi	

ID	UC2
Titolo	Disattivazione del sistema
Attori	Utente e Sistema
Eventi	1. L'utente preme il pulsante `ATTIVA` 2. Gli output sono pilotati come segue: • Le veneziane sono impostate all'angolazione di 0° • Le `Luci` sono spente e la temporizzazione è disattivata • La temperatura è impostata a 16°C ed il `Led RGB` che ne segnala il livello è impostato al colore blu
Pre-Condizioni	Il dispositivo è nello stato `ATTIVO`
Post-Condizioni	Il dispositivo si porta nello stato `INATTIVO`
Scenari Alternativi	

ID	UC3
Titolo	Configurazione del sistema
Attori	Utente e Sistema
Eventi	1. L'utente inserisce un valore di configurazione per le veneziane compreso tra 0° e 180° con risoluzione di 20° tramite interfaccia UART e preme invio. 2. L'utente inserisce un valore di configurazione per le luci: • Accese: (a/A) • Spente: (s/S) e preme invio. 3. L'utente inserisce un valore di temporizzazione per le luci: • Inattiva: 0 • Attiva: valore compreso tra 5 e 30 secondi e preme invio 4. L'utente inserisce un valore di temperatura compreso tra 16°C e 30°C e preme invio 5. Il sistema imposta gli output sulla base dei valori di configurazione inseriti ai passi precedenti
Pre-Condizioni	Il sistema è stato avviato per la prima volta oppure il sistema si trova modalità di `FUNZIONAMENTO` e viene premuto il tasto `CONFIG`
Post-Condizioni	Se tutti i valori inseriti sono validi la configurazione viene conclusa ed il sistema si porta in modalità `FUNZIONAMENTO`
Scenari Alternativi	• In caso di inserimento errato di un valore di configurazione: 1. Il sistema tramite interfaccia UART mostra un messaggio di errore 2. L'utente può reinserire il valore di configurazione • L'utente preme il pulsante `CONFIG`: 1. Il sistema esce dalla modalità di configurazione solo se è già presente una configurazione valida *(Il sistema non è stato appena avviato)*

ID	UC4
Titolo	Attivazione manuale delle luci
Attori	Utente e Sistema
Eventi	1. L'utente preme il pulsante `LUCI`
Pre-Condizioni	Il dispositivo è nello stato `ATTIVO` ed in modalità `FUNZIONAMENTO` e le luci sono `SPENTE`
Post-Condizioni	le luci sono `ACCESE`
Scenari Alternativi	

ID	UC5
Titolo	Disattivazione manuale delle luci
Attori	Utente e Sistema
Eventi	1. L'utente preme il pulsante `LUCI`
Pre-Condizioni	Il dispositivo è nello stato `ATTIVO` ed in modalità `FUNZIONAMENTO` e le luci sono `ACCESE`
Post-Condizioni	le luci sono `SPENTE`
Scenari Alternativi	

ID	UC6
Titolo	Attivazione manuale delle luci temporizzate
Attori	Utente e Sistema
Eventi	1. L'utente preme il pulsante `LUCI TEMP`
Pre-Condizioni	Il dispositivo è nello stato `ATTIVO` ed in modalità `FUNZIONAMENTO` e le luci sono `SPENTE` oppure `ACCESE` in modalità non temporizzata
Post-Condizioni	le luci sono `ACCESE` per il periodo di temporizzazione impostato durante la fase di `CONFIGURAZIONE`
Scenari Alternativi	

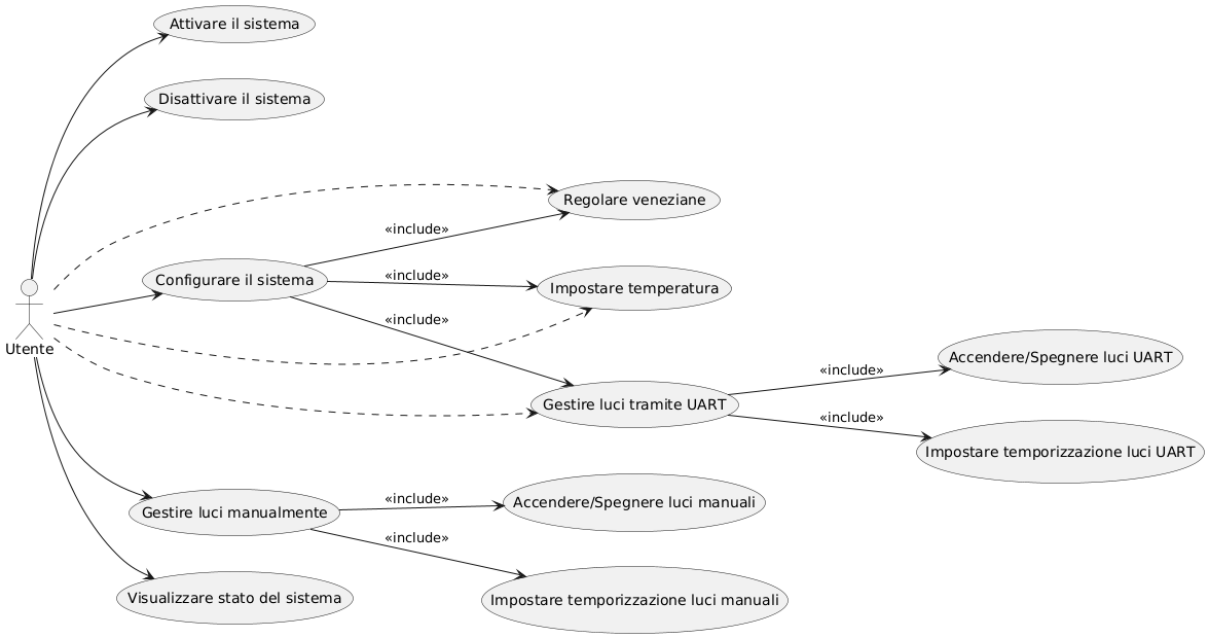


Figura 1: Use case diagram

4 Progettazione

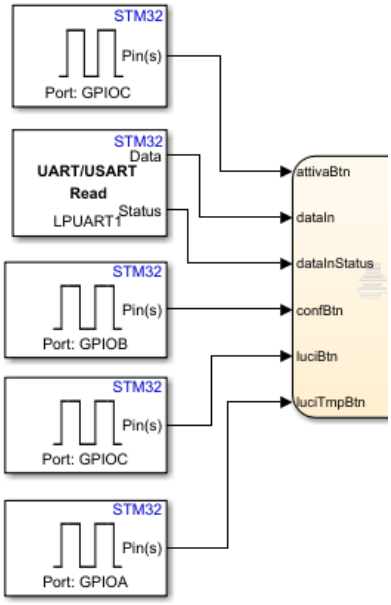
In questa sezione viene descritta la progettazione del sistema, comprendente la definizione della macchina a stati finiti e la modellazione delle logiche di controllo in Stateflow.

4.1 L'ambiente di progettazione `Simulink Stateflow`

Abbiamo utilizzato `Simulink Stateflow` come ambiente di modellazione e simulazione per rappresentare la logica di controllo del nostro sistema. Questo strumento ci è stato particolarmente utile perché ci ha consentito di progettare e simulare il comportamento del sistema attraverso macchine a stati finiti, definendo stati, transizioni, eventi e azioni associate.

Grazie a Stateflow, abbiamo potuto verificare in simulazione il funzionamento corretto della logica prima di implementarla sull'hardware, gestire temporizzazioni e condizioni complesse, e coordinare dispositivi diversi come veneziane, luci e riscaldamento. Inoltre, la possibilità di generare automaticamente codice C ci ha permesso di integrare facilmente le logiche modellate sulla scheda STM32, accelerando la fase di sviluppo e test.

4.2 Input del sistema



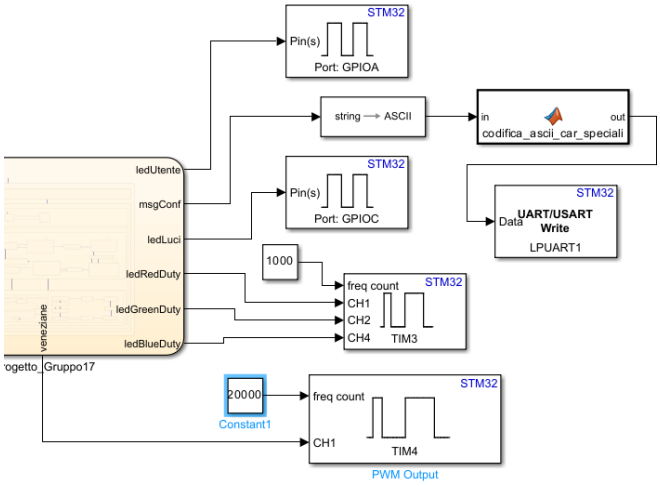
1. **attivaBtn**: pulsante dedicato all'attivazione e alla disattivazione del sistema. È collegato al pin `PC13` della porta `GPIOC` ed è configurato come `input` con resistenza di `pull-down` interna.
2. **confBtn**: pulsante per l'accesso e l'uscita dalla modalità configurazione. È collegato al pin `PB0` della porta `GPIOB`, configurato come `input` con resistenza di `pull-down`.
3. **luciBtn**: pulsante per il controllo on/off dell'illuminazione. È collegato al pin `PC10` della porta `GPIOC`, configurato come `input` con resistenza di `pull-down`.
4. **luciTmpBtn**: pulsante per l'attivazione temporizzata dell'illuminazione. È collegato al pin `PA15` della porta `GPIOA`, configurato come `input` con resistenza di `pull-down`.

Figura 2: Blocchi di input del sistema in ambiente Simulink

5. Blocco UART/USART Read: Il blocco di acquisizione seriale consente l'interfacciamento tra utente e sistema tramite periferica `USART`. Il modulo è configurato per la ricezione asincrona di caratteri con **baud rate pari a 115200 bps**, garantendo un trasferimento dati affidabile e compatibile con i dispositivi di comunicazione standard. In uscita fornisce due segnali principali:

- **dataIn**: carattere ricevuto dall'interfaccia seriale, rappresentato in formato `ASCII`.
- **dataInStatus**: segnale di stato (flag) che indica l'esito dell'operazione di lettura.

4.3 Output del sistema



1. **ledUtente**: LED dedicato all'indicazione dello stato operativo del sistema (attivo/inattivo). È collegato al pin `PA5` della porta `GPIOA`, configurato come `output push-pull`.
2. **ledLuci**: uscita digitale destinata al pilotaggio del relè di attivazione dell'illuminazione. È collegato al pin `PC2` della porta `GPIOC`, configurato come `output push-pull`.

Figura 3: Blocchi di output del sistema e relative configurazioni GPIO/PWM

3. **ledRedDuty**, **ledGreenDuty**, **ledBlueDuty**: segnali di duty-cycle utilizzati per il pilotaggio dei tre canali RGB del LED multicolore. Tali segnali sono forniti in ingresso al blocco `PWM Output 1`, che utilizza il timer `TIM3` configurato rispettivamente sui canali `CH1`, `CH2` e `CH4`.

- La costante `1000` definisce il valore di conteggio (`ARR`, Auto-Reload Register) corrispondente a una frequenza operativa di `1 kHz`, garantendo un funzionamento stabile e privo di flickering visibile.

4. **veneziane**: segnale di duty-cycle utilizzato per il pilotaggio del servomotore che regola l'angolo di apertura delle veneziane. Tale segnale è fornito in inglesso al blocco `PWM Output`, che utilizza il timer `TIM4` configurato sul canale `CH1`.

- La costante `20000` imposta il valore di conteggio del registro `ARR` (Auto-Reload Register), corrispondente a una frequenza operativa di `50 Hz`. Tale configurazione consente di generare un segnale PWM con periodo pari a `20 ms`, conforme agli standard di pilotaggio del servomotore, garantendo così un controllo stabile e preciso della posizione.

5. msgConf: Stringa di testo destinata alla trasmissione tramite interfaccia seriale `UART`. La catena di elaborazione prevede i seguenti blocchi funzionali:

- **Blocco String to ASCII**: modulo che converte ciascun carattere della stringa in ingresso nel corrispondente codice numerico secondo la codifica `ASCII`. L'output è rappresentato come un vettore di interi senza segno (`uint8`).
- **Blocco codifica_ascii_cas_speciali**: routine `MATLAB Function` che effettua il riconoscimento ed il trattamento di eventuali caratteri di escape (ad es. `\n`, `\r`, `\t`

ecc.) presenti nel vettore ASCII di ingresso, restituendo in uscita un vettore ASCII conforme.

- **Blocco UART/USART Write**: Il blocco di trasmissione seriale implementa la periferica `USART` in modalità asincrona, configurata con **baud rate pari a 115200 bps**, garantendo una comunicazione stabile e conforme agli standard seriali. Il modulo riceve in ingresso un vettore di codici `ASCII` e gestisce l'invio sequenziale dei caratteri sull'interfaccia `UART`, rendendo disponibile la stringa completa all'utente o al dispositivo remoto connesso.

4.4 Struttura del firmware in Stateflow

In questa sezione viene presentata la struttura a stati del sistema, modellata mediante l'ambiente `Stateflow`. L'architettura degli stati definisce il comportamento dinamico del sistema embedded in funzione degli eventi di ingresso e delle condizioni associate. Nei sottoparagrafi seguenti verranno descritti in dettaglio i singoli stati operativi, le rispettive azioni eseguite, nonché le transizioni abilitate, specificando le condizioni logiche e gli eventi che ne determinano l'attivazione e la disattivazione.

4.4.1 Stato SistemaInattivo

Figura 4: Stato SistemaInattivo in Stateflow

Lo stato `SistemaInattivo` rappresenta la condizione di default del sistema, ovvero la fase in cui esso risulta non operativo. In questa modalità vengono inizializzate le configurazioni predefinite per le uscite.

1. **Transizioni di ingresso:** pressione del pulsante **attivaBtn** quando il sistema è attivo
2. **Transizioni di uscita:** pressione del pulsante **attivaBtn**

Per la corretta gestione dell'evento di pressione del pulsante `attivaBtn`, sono state introdotte due variabili:

1. `attivaBtn`: variabile che rappresenta il valore logico corrente acquisito dalla linea di input GPIO associata al pulsante `attivaBtn`.
2. `attiva_btn_prev`: variabile di stato che memorizza il valore logico rilevato al ciclo di campionamento precedente. Questa variabile consente di individuare in maniera univoca la transizione da stato `PRESSED` a stato `NOTPRESSED`, permettendo di riconoscere l'evento di rilascio del pulsante.

Le costanti logiche `PRESSED` e `NOTPRESSED` sono definite a livello di sistema e corrispondono rispettivamente ai valori `1` e `0`.
La medesima logica di gestione è stata adottata per ciascun pulsante del sistema (`confBtn`, `luciBtn`, `luciTmpBtn`), garantendo uniformità e affidabilità nel riconoscimento delle pressioni utente.

4.4.2 Stato SistemaAttivo

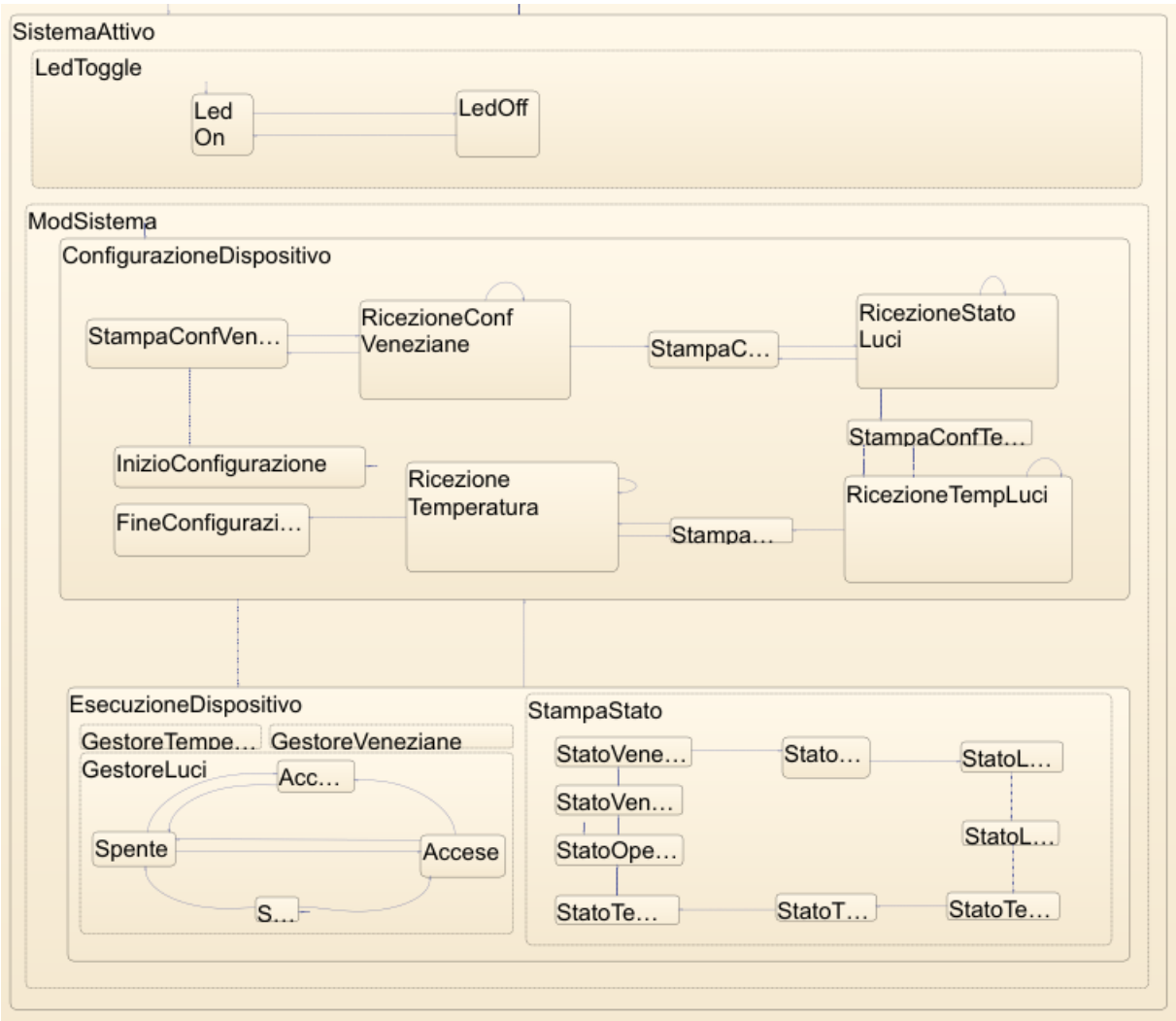


Figura 5: Stato SistemaAttivo in Stateflow

Nello stato attivo abbiamo due sottostati eseguiti in parallelo, ossia **LedToggle** e **ModSistema**. Il primo si occupa di notificare l'utente dell'attività del sistema effettuando il blinking del **ledUtente** ad una frequenza di 2hz (ogni 0.5 secondi) tramite transizione automatica tra due sottostati che accendono e spengono il led, mentre il secondo si occupa di gestire le due modalità operative del sistema, ossia **Configurazione** e **Esecuzione**

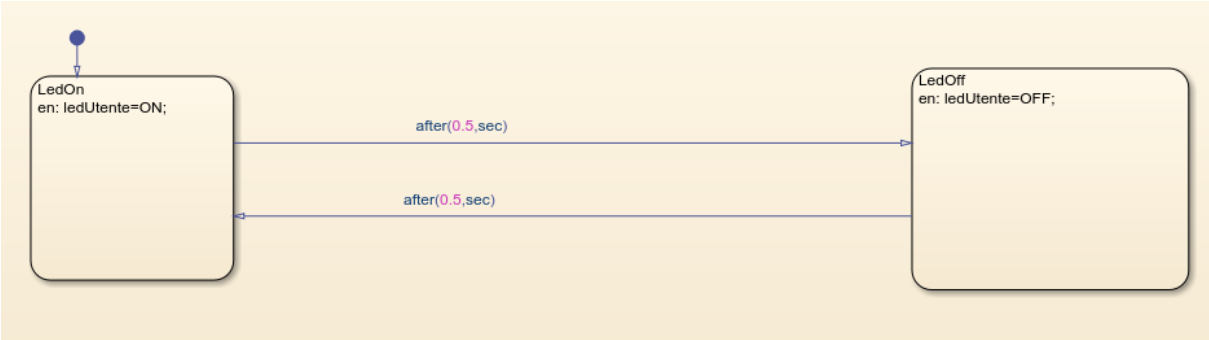


Figura 6: Stato LedToggle in Stateflow

4.4.3 Stato ConfigurazioneDispositivo

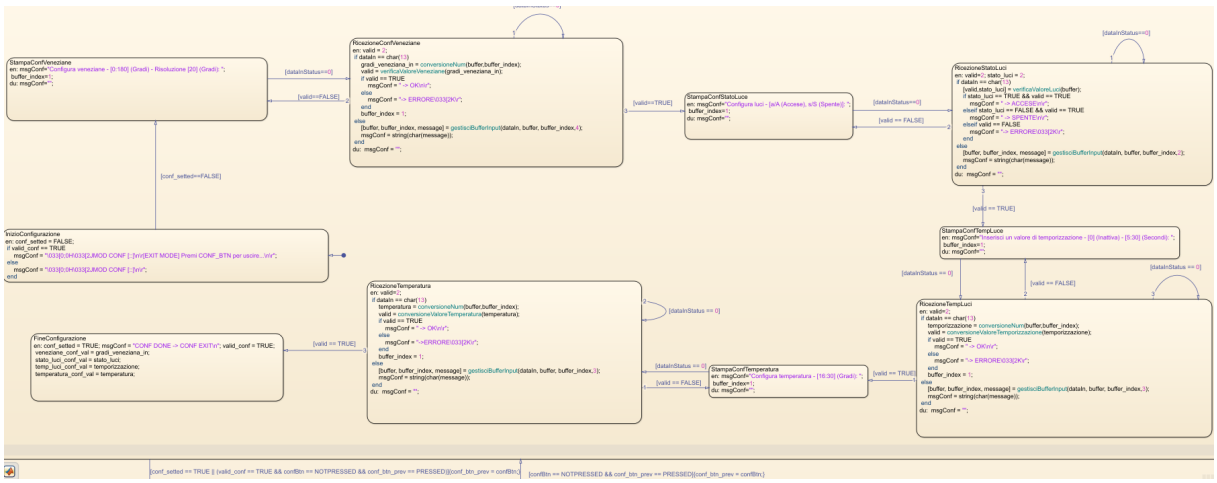


Figura 7: Stato ConfigurazioneDispositivo in Stateflow

Questo stato si occupa di gesitre la modalità operativa si **Configurazione**, comprende molti sottostati eseguiti sequenzialmente e consente di impostare correttamente tutti i parametri di configurazione.

Logica di funzionamento generale

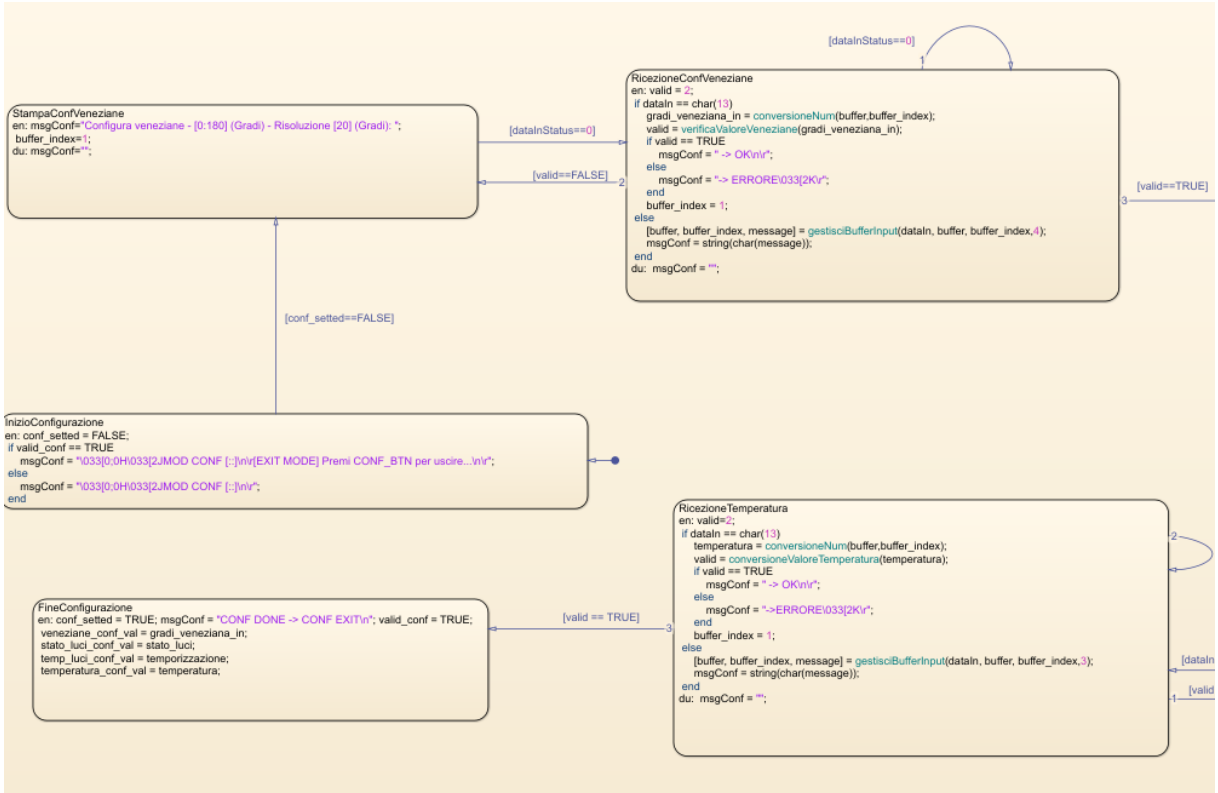


Figura 8: Logica generale dello stato ConfigurazioneDispositivo in Stateflow

Le costanti logiche `TRUE` e `FALSE` sono definite a livello di sistema e corrispondono rispettivamente ai valori `1` e `0`. La logica di richiesta e gestione dei parametri di configurazione per **luci** e **temperatura** è analoga a quella descritta per le **veneziane**.

1. **Stato InizioConfigurazione**:
   - Imposta il flag `conf_setted` a `FALSE`, indicando che la configurazione dei parametri non è ancora stata completata.
   - Trasmette un messaggio su interfaccia `UART` per notificare all'utente l'ingresso in modalità configurazione.
   - Se il flag `valid_conf` risulta `TRUE`, il sistema consente all'utente di uscire immediatamente dalla modalità configurazione, in quanto è già presente una configurazione valida.
2. **Stato StampaConfVeneziane**:
   - Trasmette un messaggio su `UART` richiedendo all'utente l'inserimento di un valore di configurazione per le veneziane.
   - Inizializza l'indice del buffer di ricezione UART (`buffer_index`) al valore `1`.

- Rimane in attesa della ricezione di un carattere dall'interfaccia `UART`.
- Verifica l'avvenuta ricezione controllando che il flag `dataInStatus` sia impostato a `0` (operazione di lettura completata).
- In caso positivo, transita verso lo stato successivo.

3. **Stato RicezioneConfVeneziane**:
   - Se il carattere ricevuto corrisponde al carattere di invio (*terminatore*):
     - Converte i valori numerici memorizzati nel buffer `buffer`.
     - Verifica la validità del valore configurato.
     - In caso di valore valido, procede verso lo stato successivo, richiedendo i parametri di configurazione restanti, impostando il flag `valid` a `TRUE`.
     - In caso di valore non valido, trasmette un messaggio di errore su `UART`, invia un carattere di escape per cancellare la riga corrente nel terminale e ritorna allo stato precedente (**StampaConfVeneziane**), impostando il flag `valid` a `FALSE`
   - Se il carattere ricevuto è diverso dal terminatore:
     - Inserisce il carattere nella prima posizione libera del buffer di ricezione.
     - Esegue l'*echo* del carattere sull'interfaccia `UART`, permettendo all'utente di visualizzare i dati immessi.
4. **Stato FineConfigurazione**:
   - Questo stato viene raggiunto quando l'utente ha completato correttamente l'inserimento di tutti i parametri di configurazione richiesti.
   - I valori acquisiti vengono memorizzati nelle seguenti variabili dedicate:
     - `veneziane_conf_val`
     - `stato_luci_conf_val`
     - `temp_luci_conf_val`
     - `temperatura_conf_val`

     Queste variabili garantiscono la persistenza della configurazione all'interno del firmware, rendendola disponibile anche per le successive attivazioni del sistema.
   - Lo stato aggiorna i flag di controllo:
     - `conf_setted = TRUE`
     - `valid_conf = TRUE`

In questo modo, ad una successiva entrata nella modalità di configurazione, l'utente potrà decidere di uscire immediatamente senza dover reinserire i parametri, poiché una configurazione valida è già disponibile.

Transizioni di Ingresso e Uscita

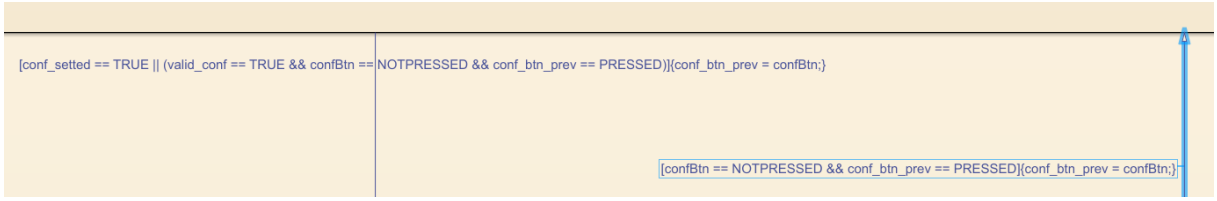


Figura 9: Transizioni di Ingresso/Uscita allo stato ConfigurazioneDispositivo in Stateflow

1. **Transizioni di Ingresso**: Le condizioni che abilitano l'entrata nello stato di configurazione sono:
   - Il sistema si trova nello stato `SistemaInattivo` e viene rilevata la pressione del pulsante `attivaBtn`.
   - Il sistema si trova nello stato `EsecuzioneDispositivo` e viene rilevata la pressione del pulsante `confBtn`.
2. **Transizioni di Uscita**: Le condizioni che abilitano l'uscita dallo stato di configurazione sono:
   - Viene rilevata la pressione del pulsante `confBtn` e il flag `valid_conf` risulta impostato a `TRUE`, condizione che indica la presenza di una configurazione già valida.
   - Il flag `conf_setted` risulta impostato a `TRUE`, condizione che segnala il completamento con successo di una nuova configurazione.

4.4.4 Stato EsecuzioneDispositivo

Questo stato rappresenta la modalità operativa principale del sistema ed è il cuore del comportamento run-time. Al suo interno sono definiti diversi sottostati eseguiti in parallelo, che consentono la gestione concorrente delle varie funzionalità del dispositivo. Durante l'esecuzione, il sistema elabora gli input provenienti sia dai pulsanti utente sia dalle variabili interne di stato e, in base ai parametri di configurazione precedentemente impostati, effettua la scrittura dei valori sugli output delle periferiche hardware, quali GPIO, PWM e UART. In tal modo, le configurazioni definite dall'utente e lo stato corrente del sistema vengono tradotti in azioni concrete sui dispositivi controllati. L'organizzazione in sottostati paralleli permette inoltre di mantenere indipendenti le diverse componenti operative, come la gestione delle veneziane, il controllo delle luci e la regolazione della temperatura, garantendo una risposta coerente e reattiva alle sollecitazioni esterne.

Stati GestoreTemperature/Veneziane/Luci

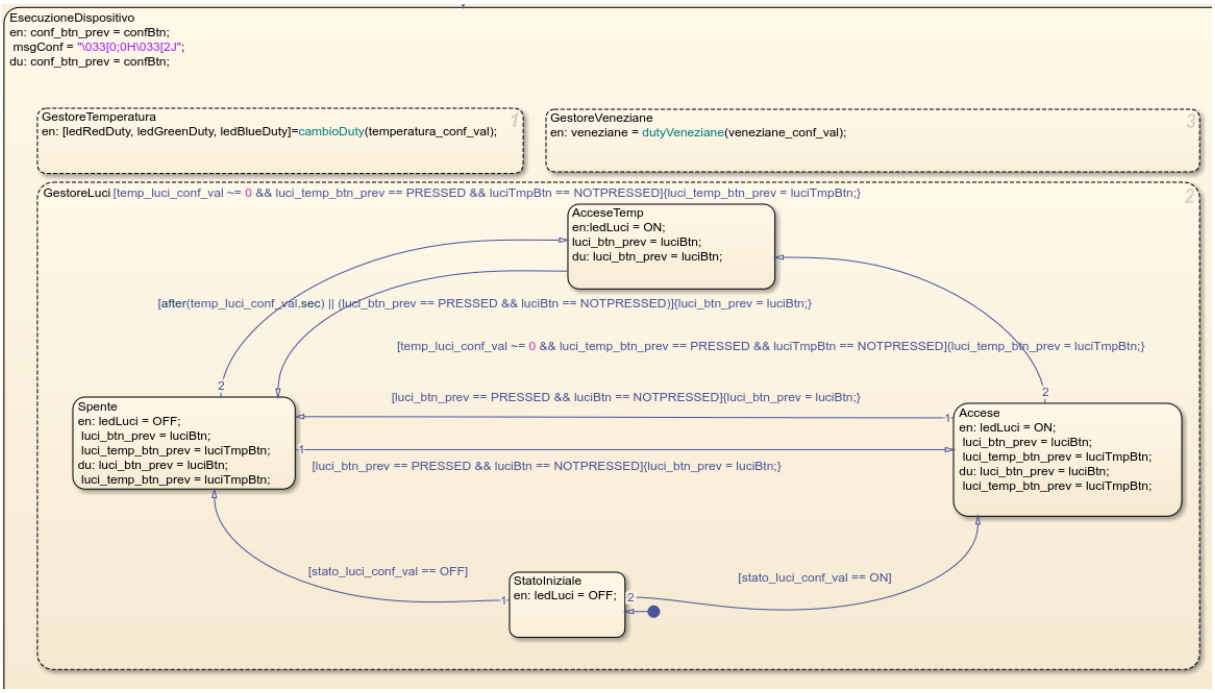


Figura 10: Stati GestoreTemperature/Veneziane/Luci in Stateflow

Negli stati **GestoreTemperatura** e **GestoreVeneziane** vengono impiegate due routine di tipo `Matlab Function`, che prendono in input i valori di configurazione rispettivi e calcolano i segnali di controllo per gli attuatori associati:

1. **ledRedDuty, ledGreenDuty, ledBlueDuty:** duty-cycle dei canali RGB del LED multicolore, calcolati in funzione del valore di temperatura configurato, al fine di riprodurre visivamente lo stato termico desiderato.

2. **veneziane:** duty-cycle del segnale PWM fornito al servomotore responsabile della regolazione dell'angolo di apertura delle veneziane, determinato sulla base del parametro di configurazione definito dall'utente.

Nello stato **GestoreLuci** sono presenti quattro stato eseguiti sequenzialmente, Le costanti logiche `ON` e `OFF` sono definite a livello di sistema e corrispondono rispettivamente ai valori `1` e `0`.

1. **Stato Iniziale:**
   - Imposta il valore di output da fornire al relè su `OFF`.
   - Viene utilizzato come stato iniziale, dal quale il sistema passa successivamente allo stato **Accese** oppure **Spente**, in base alla configurazione effettuata.
2. **Stato Accese:**
   - Imposta il valore di output da fornire al relè su `ON`.
   - Alla pressione del pulsante `luciBtn`, consente il passaggio allo stato **Spente**.
   - Alla pressione del pulsante `luciTmpBtn`, permette di passare allo stato **AcceseTmp**, solo se il valore di temporizzazione definito in fase di configurazione è diverso da 0.
3. **Stato Spente:**
   - Imposta il valore di output da fornire al relè su `OFF`.
   - Alla pressione del pulsante `luciBtn`, consente il passaggio allo stato **Accese**.
   - Alla pressione del pulsante `luciTmpBtn`, permette di passare allo stato **AcceseTmp**, solo se il valore di temporizzazione definito in fase di configurazione è diverso da 0.
4. **Stato AcceseTmp:**
   - Imposta il valore di output da fornire al relè su `ON`, per la durata specificata in fase di configurazione.
   - Alla pressione del pulsante `luciBtn`, consente il passaggio allo stato **Spente**.
   - Allo scadere del tempo di temporizzazione, il sistema ritorna automaticamente allo stato **Spente**.

Stato StampaStato

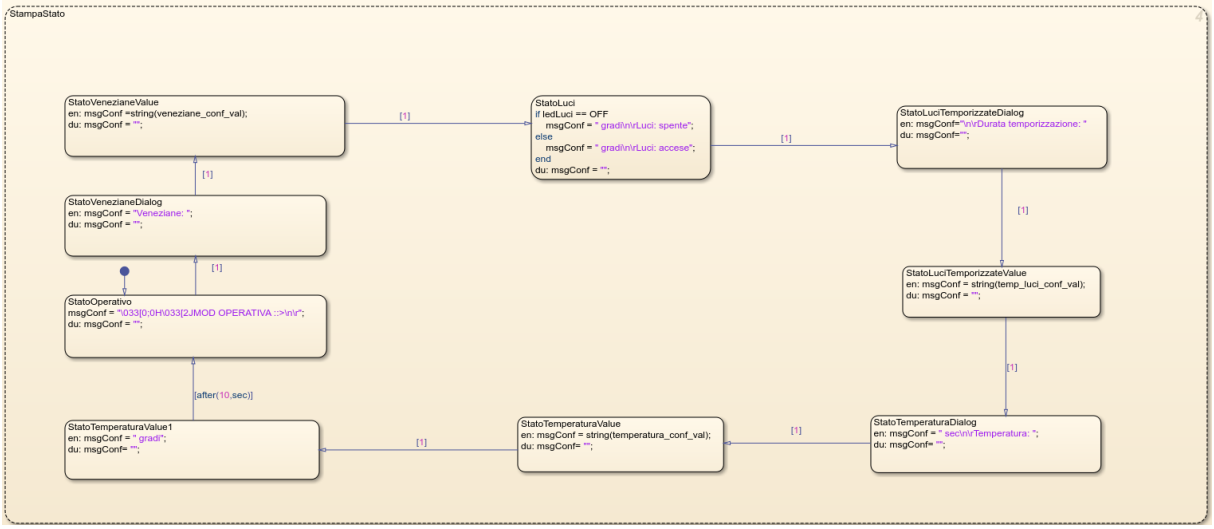


Figura 11: Stato StampaStato in Stateflow

Questo stato esegue la stampa dello stato interno del sistema sull'interfaccia UART a intervalli di 10 secondi, mentre il sistema si trova in modalità operativa **Esecuzione**. È composto da una sequenza di stati che provvedono a stampare le informazioni relative allo stato di ciascun attuatore e ai valori di configurazione attualmente impostati. Al termine di un ciclo di stampa, il sistema passa a uno stato terminale, nel quale attende 10 secondi prima di inviare un comando di escape per pulire lo schermo del terminale ed eseguire nuovamente il ciclo di stampa.

4.5 Funzioni Matlab Implementate

In questa sezione vengono analizzate le funzioni `MATLAB` implementate per supportare le funzionalità richieste dal sistema. Ciascuna funzione sarà descritta in dettaglio nelle sottosezioni successive, evidenziandone gli input, gli output e il ruolo operativo all'interno della logica complessiva del sistema embedded.

4.5.1 Funzione `verificaValoreLuci`

La funzione `verificaValoreLuci` ha lo scopo di verificare la validità di un valore ricevuto in ingresso tramite il buffer `buff` e di determinarne l'associazione con lo stato delle luci.

In particolare, la funzione analizza il primo carattere della variabile `buff`:

- Se il carattere corrisponde alla lettera **'A'** (codice ASCII `65`) o **'a'** (codice ASCII `97`), il valore è considerato valido (`valido = 1`) e le luci vengono impostate come accese (`luci = 1`).

- Se il carattere corrisponde alla lettera **'S'** (codice ASCII `83`) o **'s'** (codice ASCII `115`), il valore è considerato valido (`valido = 1`) e le luci vengono impostate come spente (`luci = 0`).
- In tutti gli altri casi, l'ingresso è considerato non valido (`valido = 0`) e la variabile `luci` assume il valore `2`, che rappresenta una condizione di errore o valore non riconosciuto.

L'output della funzione è quindi costituito da due variabili:

- `valido`: flag che indica se il carattere letto è valido o meno;
- `luci`: stato logico associato alle luci (`1 = accese`, `0 = spente`, `2 = non valido`).

4.5.2 Funzione `conversioneNum`

La funzione `conversioneNum(buff, buff_index)` prende in ingresso un buffer di caratteri e un indice che ne rappresenta la lunghezza. Il suo scopo è convertire la sequenza di caratteri numerici contenuti nel buffer in un valore numerico intero non negativo.

La funzione procede carattere per carattere:

- Se il carattere analizzato è compreso tra i codici ASCII delle cifre `0` (48) e `9` (57), il valore `val` viene aggiornato moltiplicandolo per 10 e sommando la cifra corrispondente.
- Se viene trovato un carattere non numerico, la funzione interrompe la conversione e restituisce il valore `255`, a indicare un errore di conversione.

L'output della funzione è:

- `val`: numero intero non negativo ottenuto dalla conversione dei caratteri numerici, oppure 255 se l'ingresso non è valido.

4.5.3 Funzione `verificaValoreVeneziane`

La funzione `verificaValoreVeneziane(gradi)` verifica la validità di un valore di angolo per l'apertura delle veneziane. In particolare, il valore è considerato valido se:

- è compreso tra 0° e 180° inclusi;
- è un multiplo di 20.

L'output è:

- `valido = 1` se il valore è accettabile;
- `valido = 0` in caso contrario.

4.5.4 Funzione `conversioneValoreTemporizzazione`

La funzione `conversioneValoreTemporizzazione(temp)` controlla se il valore di temporizzazione inserito è valido. Le condizioni di validità sono:

- il valore deve essere compreso tra 5 e 30 secondi;
- in alternativa, può assumere il valore 0.

L'output è:

- `valido = 1` se il valore soddisfa le condizioni;
- `valido = 0` altrimenti.

4.5.5 Funzione `gestisciBufferInput`

La funzione `gestisciBufferInput(d, buff, buff_index, dimensione)` gestisce l'aggiornamento del buffer di input in base ai caratteri ricevuti. Le operazioni principali sono:

- Se l'indice attuale del buffer è minore della dimensione massima e il carattere ricevuto non è il tasto `Backspace` (codice ASCII 127), il carattere viene aggiunto al buffer, incrementando `buff_index`.
- Se il carattere ricevuto è `Backspace` e l'indice è maggiore di 1, l'ultimo carattere inserito viene eliminato decrementando `buff_index`.

Gli output della funzione sono:

- `buff`: buffer aggiornato;
- `buff_index`: indice aggiornato;
- `msg`: ultimo carattere processato.

4.5.6 Funzione `conversioneValoreTemperatura`

La funzione `conversioneValoreTemperatura(temp)` verifica la validità di un valore di temperatura. La condizione di validità è che la temperatura sia compresa tra 16°C e 30°C inclusi.

L'output è:

- `valido = 1` se il valore è compreso nell'intervallo;
- `valido = 0` altrimenti.

4.5.7 Funzione `cambioDuty`

La funzione `cambioDuty(temp)` calcola i valori di duty cycle per tre componenti RGB in base alla temperatura impostata. L'idea è associare a intervalli di temperatura differenti una colorazione specifica:

- 16–18 °C: colore blu (`duty_blue` alto, altri bassi).
- 19–21 °C: colore viola (rosso e blu al 50%).
- 22–24 °C: colore giallo (rosso alto, verde basso).
- 25–27 °C: colore arancione (rosso alto, verde molto basso).
- $\geq$ 28 °C: colore rosso (solo componente rossa al 100%).

Gli output sono tre valori interi che rappresentano le percentuali di duty cycle per:

- `duty_red`, `duty_green`, `duty_blue`.

4.5.8 Funzione `dutyVeneziane`

La funzione `dutyVeneziane(gradi_ven)` calcola l'impulso PWM da inviare al servomotore che controlla l'orientamento delle veneziane. Il calcolo è effettuato mediante la formula:

$$impulso = 500 + \frac{gradi \cdot (2500 - 500)}{180}$$

dove `gradi` rappresenta l'angolo di apertura delle veneziane.

L'output è:

- `impulso`: valore numerico corrispondente all'ampiezza dell'impulso da applicare al servo.

4.5.9 Funzione `codifica_ascii_car_speciali`

La funzione `codifica_ascii_car_speciali(in)` ha lo scopo di convertire una sequenza di caratteri di input in una sequenza di codici ASCII, gestendo in particolare le sequenze di escape e i caratteri speciali.

Input:

- `in`: array di caratteri da codificare.

Output:

- `out`: array di valori interi non negativi (`uint8`) contenente i codici ASCII risultanti dalla codifica.

Logica di funzionamento:

- La funzione prealloca un vettore `out` di lunghezza massima fissata (`MAX_LEN = 100`) per contenere il risultato.
- Scorre l'input carattere per carattere (`i`), gestendo i seguenti casi:
  - **Sequenze di escape specifiche:**
    - Riconosce la sequenza di caratteri `\033[0;0H\033[2J` e la sostituisce con la corrispondente sequenza di byte che corrisponde a clear screen.
    - Riconosce la sequenza `\033[2K` e la sostituisce con la sequenza di byte che corrisponde a clear line.
  - **Escape dei caratteri singoli:** se incontra il simbolo `\`, interpreta il carattere successivo per generare il corrispondente codice ASCII:
    - `\n` → 10 (newline);
    - `\r` → 13 (carriage return);
    - `\t` → 9 (tabulazione);
    - `\\` → 92 (backslash);
    - `\'` → 39 (apostrofo);
    - `\"` → 34 (virgolette).
  - Se il carattere dopo `\` non è riconosciuto, vengono copiati sia il backslash (92) sia il carattere stesso.
  - **Caratteri normali:** se non rientrano nei casi precedenti, vengono copiati direttamente nel buffer `out`.
- L'output viene costruito progressivamente fino al raggiungimento della lunghezza massima supportata (`MAX_LEN`).
- Al termine, la funzione stampa a video il contenuto del vettore `out`.